

Object-Oriented Programming in Java

A Comprehensive Guide

Slide 1: Introduction to OOP in Java

What is Object-Oriented Programming?

- Programming paradigm centered around objects
 - Objects combine data (attributes) and behavior (methods)
 - Java is a pure object-oriented language
 - Everything in Java is part of a class (except primitives)
-

Slide 2: Why Use OOP in Java?

Benefits:

- **Encapsulation** - Data hiding and protection
 - **Inheritance** - Code reusability through parent-child relationships
 - **Polymorphism** - Multiple forms of methods/objects
 - **Abstraction** - Hide complex implementation details
 - **Modularity** - Organized, maintainable code structure
-

Slide 3: Java Classes - The Foundation

What is a Class?

- Blueprint or template for creating objects
- Defines structure (attributes) and behavior (methods)
- Every Java program must have at least one class

Basic Syntax:

```
public class ClassName {  
    // class body  
}
```

Slide 4: Defining a Simple Class

```
public class Dog {  
    // Class body is empty for now  
}
```

Creating an object:

```
Dog myDog = new Dog();
```

- `Dog` is the class (blueprint)
- `myDog` is an object/instance of `Dog`
- `new` keyword creates the object in memory

Slide 5: Class Structure Overview

```
public class Student {  
    // 1. Attributes (fields/variables)  
    private String name;  
    private int age;  
  
    // 2. Constructor  
    public Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    // 3. Methods  
    public void study() {  
        System.out.println(name + " is studying");  
    }  
}
```

Slide 6: Defining Attributes (Fields)

Attributes = Data stored in objects

```
public class Car {  
    // Instance variables (attributes)  
    private String brand;  
    private String model;  
    private int year;  
    private double price;  
}
```

Key Points:

- Declared inside class but outside methods
 - Each object has its own copy
 - Use access modifiers (private, public, protected)
-

Slide 7: Attribute Types

1. Instance Variables:

```
public class Person {  
    private String name; // Each person has unique name  
    private int age;      // Each person has unique age  
}
```

2. Class Variables (Static):

```
public class Person {  
    private static int population = 0; // Shared by all Person objects  
}
```

Slide 8: Initializing Attributes

Three ways to initialize:

```
public class Book {  
    // 1. Direct initialization  
    private int pages = 100;  
  
    // 2. Constructor initialization  
    private String title;  
  
    // 3. Initialization block  
    private String author;  
    {  
        author = "Unknown";  
    }  
  
    public Book(String title) {  
        this.title = title; // Constructor  
    }  
}
```

Slide 9: Constructors

Special method to initialize objects:

```
public class Dog {  
    private String name;  
    private int age;  
  
    // Constructor  
    public Dog(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

Rules:

- Same name as class
 - No return type (not even void)
 - Called automatically when object is created
-

Slide 10: Multiple Constructors (Overloading)

```
public class Dog {  
    private String name;  
    private int age;  
  
    // Constructor 1: No parameters  
    public Dog() {  
        this.name = "Unknown";  
        this.age = 0;  
    }  
  
    // Constructor 2: With parameters  
    public Dog(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    // Constructor 3: Only name  
    public Dog(String name) {  
        this.name = name;  
        this.age = 0;  
    }  
}
```

Slide 11: The **this** Keyword

Refers to current object:

```
public class Person {  
    private String name;  
  
    public Person(String name) {  
        this.name = name; // this.name = instance variable  
                           // name = parameter  
    }  
  
    public void printName() {  
        System.out.println(this.name); // Can use this  
        System.out.println(name);      // Or omit it  
    }  
}
```

Slide 12: Defining Methods

Methods = Actions/behaviors of objects

```
public class Calculator {  
    // Method with return type  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    // Method without return (void)  
    public void displayResult(int result) {  
        System.out.println("Result: " + result);  
    }  
}
```

Syntax:

```
accessModifier returnType methodName(parameters) {  
    // method body  
}
```

Slide 13: Method Components

```
public int calculateSum(int x, int y) {  
    int sum = x + y;  
}
```

```
    return sum;
}
```

Parts:

- `public` - Access modifier
- `int` - Return type
- `calculateSum` - Method name
- `(int x, int y)` - Parameters
- `return sum` - Return statement
- Method body - Code inside `{}`

Slide 14: Method Types

1. Instance Methods:

```
public class Dog {
    public void bark() {
        System.out.println("Woof!");
    }
}
```

2. Static Methods:

```
public class Math {
    public static int add(int a, int b) {
        return a + b;
    }
}
```

Slide 15: Calling Methods

```
public class Dog {
    private String name;

    public Dog(String name) {
        this.name = name;
    }

    public void bark() {
        System.out.println(name + " says Woof!");
    }
}
```

```
// Main class
public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog("Buddy");
        myDog.bark(); // Output: Buddy says Woof!
    }
}
```

Slide 16: Calling Static vs Instance Methods

```
public class Example {
    // Instance method
    public void instanceMethod() {
        System.out.println("Instance method");
    }

    // Static method
    public static void staticMethod() {
        System.out.println("Static method");
    }
}

// Calling methods
Example obj = new Example();
obj.instanceMethod();           // Need object
Example.staticMethod();         // Call on class directly
```

Slide 17: Access Modifiers - Overview

Modifier	Class	Package	Subclass	World
public	✓	✓	✓	✓
protected	✓	✓	✓	X
default (no modifier)	✓	✓	X	X
private	✓	X	X	X

Slide 18: Public Access Modifier

Accessible from anywhere:

```
public class Person {
    public String name; // Public attribute

    public void greet() { // Public method
    }
```

```
        System.out.println("Hello!");
    }
}

// Another class
public class Main {
    public static void main(String[] args) {
        Person person = new Person();
        person.name = "Alice"; // Accessible
        person.greet();        // Accessible
    }
}
```

Slide 19: Private Access Modifier

Only accessible within the same class:

```
public class BankAccount {
    private double balance; // Private attribute

    private void calculateInterest() { // Private method
        // Only accessible inside this class
    }

    public void deposit(double amount) {
        balance += amount; // Private attribute accessed internally
    }
}

// Another class
BankAccount account = new BankAccount();
// account.balance = 1000; // ERROR: Cannot access
// account.calculateInterest(); // ERROR: Cannot access
account.deposit(1000); // OK: Public method
```

Slide 20: Protected Access Modifier

Accessible in same package and subclasses:

```
public class Animal {
    protected String species; // Protected attribute

    protected void move() { // Protected method
        System.out.println("Moving...");
    }
}
```



```
public class Dog extends Animal {
    public void display() {
        System.out.println(species); // Accessible in subclass
        move();                      // Accessible in subclass
    }
}
```

Slide 21: Default (Package-Private) Access

No modifier = accessible only in same package:

```
class Helper { // Default class access
    int value; // Default attribute

    void assist() { // Default method
        System.out.println("Helping...");
    }
}

// Same package: OK
// Different package: ERROR
```

Slide 22: Encapsulation with Getters and Setters

```
public class Person {
    private String name; // Private attribute
    private int age;

    // Getter
    public String getName() {
        return name;
    }

    // Setter
    public void setName(String name) {
        this.name = name;
    }

    // Getter with validation
    public int getAge() {
        return age;
    }

    // Setter with validation
    public void setAge(int age) {
        if (age > 0 && age < 150) {
            this.age = age;
        }
    }
}
```

```
    }  
  }  
}
```

Slide 23: Why Use Private + Getters/Setters?

Benefits of Encapsulation:

1. **Data Protection:** Prevent invalid values

```
public void setAge(int age) {  
    if (age > 0) this.age = age;  
}
```

2. **Flexibility:** Change implementation without affecting users
3. **Read-Only/Write-Only:** Control access

```
// Read-only: Only getter, no setter  
public String getId() {  
    return id;  
}
```

4. **Validation:** Add business logic

Slide 24: Complete Example - Student Class

```
public class Student {  
    // Private attributes  
    private String name;  
    private int rollNumber;  
    private double gpa;  
  
    // Static attribute  
    private static int totalStudents = 0;  
  
    // Constructor  
    public Student(String name, int rollNumber) {  
        this.name = name;  
        this.rollNumber = rollNumber;  
        this.gpa = 0.0;  
        totalStudents++;  
    }  
  
    // Getters and Setters  
    public String getName() {
```

```
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getGpa() {
        return gpa;
    }

    public void setGpa(double gpa) {
        if (gpa >= 0.0 && gpa <= 4.0) {
            this.gpa = gpa;
        }
    }
}
```

Slide 25: Student Class - Methods

```
// Instance method
public void study(String subject) {
    System.out.println(name + " is studying " + subject);
}

// Instance method
public boolean isPassing() {
    return gpa >= 2.0;
}

// Static method
public static int getTotalStudents() {
    return totalStudents;
}

// Display method
public void displayInfo() {
    System.out.println("Name: " + name);
    System.out.println("Roll Number: " + rollNumber);
    System.out.println("GPA: " + gpa);
}
}
```

Slide 26: Using the Student Class

```
public class Main {
    public static void main(String[] args) {
        // Create objects
    }
}
```

```
Student student1 = new Student("Alice", 101);
Student student2 = new Student("Bob", 102);

// Set attributes using setters
student1.setGpa(3.8);
student2.setGpa(2.5);

// Call instance methods
student1.study("Mathematics");
student2.study("Physics");

// Get attributes using getters
System.out.println(student1.getName() + "'s GPA: " + student1.getGpa());

// Call static method
System.out.println("Total Students: " + Student.getTotalStudents());

// Display info
student1.displayInfo();
}
}
```

Slide 27: Method Overloading

Multiple methods with same name, different parameters:

```
public class Calculator {
    // Add two integers
    public int add(int a, int b) {
        return a + b;
    }

    // Add three integers
    public int add(int a, int b, int c) {
        return a + b + c;
    }

    // Add two doubles
    public double add(double a, double b) {
        return a + b;
    }
}

Calculator calc = new Calculator();
System.out.println(calc.add(5, 3));           // 8
System.out.println(calc.add(5, 3, 2));        // 10
System.out.println(calc.add(5.5, 3.2));       // 8.7
```

Slide 28: Best Practices

1. Naming Conventions:

- Classes: `PascalCase` (ClassName)
- Methods: `camelCase` (methodName)
- Constants: `UPPER_SNAKE_CASE` (MAX_VALUE)

2. Encapsulation:

- Make attributes private
- Provide public getters/setters

3. Single Responsibility:

- Each class should have one clear purpose

4. Meaningful Names:

- `getUserName()` not `get()`
- `calculateTotal()` not `calc()`

Slide 29: Common Mistakes to Avoid

✗ Making everything public:

```
public class Person {  
    public String name; // Bad: Direct access  
    public int age;  
}
```

✓ Use private with getters/setters:

```
public class Person {  
    private String name; // Good: Encapsulated  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
}
```

✗ Forgetting `this` keyword:

```
public Person(String name) {  
    name = name; // Both refer to parameter!  
}
```

Slide 30: Practical Example - BankAccount

```
public class BankAccount {
    private String accountNumber;
    private String ownerName;
    private double balance;

    public BankAccount(String accountNumber, String ownerName) {
        this.accountNumber = accountNumber;
        this.ownerName = ownerName;
        this.balance = 0.0;
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: $" + amount);
        }
    }

    public boolean withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: $" + amount);
            return true;
        }
        System.out.println("Insufficient funds");
        return false;
    }
}
```

Slide 31: BankAccount - Complete

```
public double getBalance() {
    return balance;
}

public String getAccountNumber() {
    return accountNumber;
}

public String getOwnerName() {
    return ownerName;
}

public void displayAccountInfo() {
    System.out.println("Account Number: " + accountNumber);
    System.out.println("Owner: " + ownerName);
    System.out.println("Balance: $" + balance);
}
```

```
}  
}
```

Slide 32: Using BankAccount

```
public class Main {  
    public static void main(String[] args) {  
        BankAccount account = new BankAccount("123456", "Alice");  
  
        account.deposit(1000);  
        account.withdraw(250);  
        account.withdraw(800); // Insufficient funds  
  
        System.out.println("Current Balance: $" + account.getBalance());  
        account.displayAccountInfo();  
    }  
}  
  
// Output:  
// Deposited: $1000.0  
// Withdrawn: $250.0  
// Insufficient funds  
// Current Balance: $750.0  
// Account Number: 123456  
// Owner: Alice  
// Balance: $750.0
```

Slide 33: Static vs Instance - Summary

Instance Members:

- Belong to objects
- Need object to access
- Each object has its own copy

```
Dog dog = new Dog();  
dog.bark(); // Instance method
```

Static Members:

- Belong to class
- Accessed via class name
- Shared by all objects

```
Math.sqrt(16); // Static method
Student.getTotalStudents(); // Static method
```

Slide 34: Access Modifiers - When to Use

Use **private**:

- Attributes (almost always)
- Helper methods used only internally

Use **public**:

- Methods that form the class interface
- Constants that should be accessible

Use **protected**:

- Members that subclasses need access to

Use **default (no modifier)**:

- Package-level utilities
-

Slide 35: Key Concepts Summary

Classes:

- Defined with **class** keyword
- Blueprint for creating objects

Attributes:

- Variables that hold object data
- Instance variables (per object)
- Static variables (shared)

Methods:

- Functions that define object behavior
- Instance methods (work with object data)
- Static methods (work with class data)

Constructors:

- Special methods to initialize objects
 - Same name as class, no return type
-

Slide 36: Encapsulation Benefits

Data Hiding:

- Private attributes protect data
- Public methods control access

Validation:

- Setters can validate input
- Prevent invalid states

Flexibility:

- Change implementation without affecting users
- Add logging, caching, etc.

Security:

- Control who can read/write data
 - Implement authorization logic
-

Slide 37: Real-World Analogy

Class = Car Blueprint

- Attributes: color, model, year, speed
- Methods: accelerate(), brake(), turnLeft()

Object = Actual Car

```
Car myCar = new Car("Toyota", "Red", 2023);  
myCar.accelerate();  
myCar.brake();
```

Each car (object) has:

- Its own color, model, year (attributes)
 - Same abilities: accelerate, brake (methods)
-

Slide 38: Memory Representation

```
Dog dog1 = new Dog("Buddy", 3);  
Dog dog2 = new Dog("Max", 5);
```

Memory:

Stack:	Heap:
dog1 ----->	Dog Object 1 - name: "Buddy" - age: 3
dog2 ----->	Dog Object 2 - name: "Max" - age: 5

Each object has separate memory space for attributes.

Slide 39: Constructor Chaining

```
public class Person {
    private String name;
    private int age;
    private String city;

    // Constructor 1
    public Person() {
        this("Unknown", 0, "Unknown");
    }

    // Constructor 2
    public Person(String name) {
        this(name, 0, "Unknown");
    }

    // Constructor 3 (main)
    public Person(String name, int age, String city) {
        this.name = name;
        this.age = age;
        this.city = city;
    }
}
```

Slide 40: Conclusion

Java OOP Fundamentals:

✓ **Classes** - Blueprints for objects ✓ **Attributes** - Data stored in objects ✓ **Methods** - Behaviors/actions of objects ✓ **Constructors** - Initialize objects ✓ **Access Modifiers** - Control visibility ✓ **Encapsulation** - Protect data with private + getters/setters

Remember:

- Keep attributes private

- Use meaningful names
 - Follow Java naming conventions
 - Validate input in setters
 - Design classes with single responsibility
-

Thank You!

Questions?